

LEARNING MULTIMODAL LATENT MEMORY THROUGH BOOTSTRAPPING AND ADAPTATION

Anonymous authors

Paper under double-blind review

ABSTRACT

Many manipulation decisions depend jointly on what a robot has seen, the actions it has executed, and how its proprioceptive state has evolved. *Latent memory* offers a natural way to automatically compress this growing multimodal history into a bounded representation, avoiding pre-defined memory representations of keyframes, or textual summaries. Existing latent-memory methods, however, either focus primarily on visual history or receive only indirect supervision from action prediction, providing little guidance about what information should survive compression. This raises two challenges: how to efficiently encode growing heterogeneous histories, and how to learn a good memory representation to retain information useful for manipulation. We introduce **IMPRINT**, a policy-agnostic latent memory framework that addresses both memory encoding and learning: **it compresses growing multimodal histories within and across streams into a latent representation, then bootstraps the memory for preserving complete multimodal histories and tracking task-aware progress before joint adaptation for control**. This policy-independent bootstrapping allows the same memory pipeline to integrate with different robot-policy backbones. Across simulation benchmarks containing both memory-dependent and memory-free tasks and three real-world manipulation tasks, IMPRINT improves success over the base policy by **47.4%** in simulation and **50%** in the real world. Its recurrent encoding adds only **19.6 ms** per action chunk while achieving higher success and lower latency than all evaluated memory baselines. Project website: [URL].

1 INTRODUCTION

Many manipulation tasks require information that current observations cannot reveal. A robot dusting a table, for example, must recall the original locations of objects from visual history, as well as the sweeping path from its previous motion as shown in Fig. ?? . Yet many vision-language-action (VLA) models Black et al. (2024); Bjorck et al. (2025); Intelligence et al. (2025) act only on the current observation. Augmenting them with memory requires integrating dense and redundant visual, action, and proprioceptive streams while running inference under a fixed latency budget, so memory must compactly compress those streams while remaining useful for downstream manipulation.

Existing memory-augmented policies address this with explicit memory, reducing history to a curated subset of modalities, such as a window of recent frames, selected keyframes, or textual summaries Dai et al. (2026); Mark et al. (2026); Yang et al. (2026a); Zeng et al. (2026); Sridhar et al. (2026); Torne et al. (2026). These representations are interpretable and easy for a policy to consume. However, explicitly choosing what to retain imposes a predefined notion of relevance: keyframe memories discard intermediate motion, textual summaries abstract away fine-grained sensorimotor details, and bounded frame windows lose distant history.

Learned latent memory can instead compresses all histories automatically into a continuous representation, with the potential to retain richer information. Current latent methods, however, either ingest only visual history Koo et al. (2026); Shi et al. (2026) or learned primarily through downstream action supervision or representation objectives that do not explicitly require the bottleneck to preserve the heterogeneous contents of interaction history Dashora et al. (2026).

Realizing the full potential of latent memory beyond prior work therefore raises two coupled challenges. The first is *encoding*: memory must compress heterogeneous and continually growing

streams online efficiently. The second is *learning*: how to train the memory bottleneck to retain information from each stream for better action generation. Training memory from scratch jointly with the policy through action loss alone is difficult, because memory and policy learning are coupled and the policy can exploit spurious shortcuts in the demonstrations.

We present **IMPRINT**, whose design relies on two key insights to address these challenges. **For encoding, memory should compress history at two levels: within stream and across streams over time.** A pretrained video tokenizer Wan et al. (2025) removes the spatial and temporal redundancy within the video stream, reducing it to a scale comparable with actions and proprioceptive states. A learned recurrent bottleneck Yang et al. (2025) then efficiently integrates all streams into a fixed-size memory that is updated incrementally at inference.

For learning, to reduce the difficulty of optimization, our key insight is that **memory should first be bootstrapped to be complete, not ignoring any stream during learning, and task-aware, tracking how far the task has progressed**, and only then adapted jointly with policy for control. To achieve this, we train memory to reconstruct past visual features, actions, and proprioceptive states, encouraging it to retain information from each stream. We also supervise it to estimate temporal task progress, to compress the streams into task-relevant context. Since these objectives require neither annotations nor a policy, the learned memory can be integrated into different VLA backbones through a lightweight adapter into the action expert and then fine-tuned jointly with the policy, making **IMPRINT** a policy-agnostic memory pipeline.

Our contributions are fourfold. First, we propose **IMPRINT**, a novel latent memory framework that compresses visual, action, and proprioceptive histories into latent memory, and bootstraps it with a complete and task-aware prior for downstream control. Second, **IMPRINT** is an annotation-free, policy-agnostic pipeline that is efficient at inference. Third, we systematically analyze how memory capacity, bootstrapping objectives, and training recipes affect downstream performance. Fourth, we evaluate **IMPRINT** on RMBench Chen et al. (2026b), a customized 12-task RoboCasa benchmark Nasiriany et al. (2024) spanning memory-dependent and memory-free tasks, and three real-world tasks. **IMPRINT** improves overall success over the base VLA by **47 percentage points in simulation and 50 in the real world** while adding only 20 ms of latency per prediction, and it Pareto-dominates existing memory pipelines for VLAs in both success rate and inference time.

2 RELATED WORK

Memory Benchmarks for Robotic Manipulation. Memory has been studied in mobile manipulation, where robots must retain spatial knowledge across environments Zhang & Ma (2025); Liu et al. (2025). For tabletop manipulation, MIKASA-Robo, RMBench Chen et al. (2026b), and RoboMME Dai et al. (2026) probe diverse memory tasks, largely in controlled visual environments with simple objects and limited manipulation difficulty. ReMemBench Shah et al. (2026a) and MemMimic Gao et al. (2026) introduce more complex manipulation, but offer limited scene and background diversity. These dedicated benchmarks primarily target memory-dependent tasks; MemMimic separately evaluates non-memory RoboMimic tasks outside its suite Gao et al. (2026). More broadly, RoboDojo Chen et al. (2026a) evaluates memory alongside broader manipulation capabilities, but evaluates background and scene randomization separately in its generalization tasks. Our RoboCasa-based benchmark Nasiriany et al. (2024) jointly spans memory and non-memory tasks across diverse objects and household environments, while varying manipulation complexity.

Memory-Augmented Robot Policies. Memory has been explored across different classes of robot policies. Recurrent states and extended observation histories help visuomotor policies handle partial observability and long-horizon dependencies Shah et al. (2026a); Torne et al. (2025); Gao et al. (2026); Jiang et al. (2026). Structured memory methods retain object-centric scene graphs for exploration and control Jiang et al. (2024); Qian et al. (2026); Banwasi et al. (2026). For pretrained VLAs, existing approaches retain selected keyframes Mark et al. (2026); Yang et al. (2026a); Shah et al. (2026b); Sridhar et al. (2026), combine short-term video memory with long-term textual summaries, Torne et al. (2026), or learn latent memory representations Shi et al. (2026); Koo et al. (2026). Beyond VLAs, MemoryWAM Yang et al. (2026b) explores persistent memory in world-action models, while MaP-WAM Zhao et al. (2026) converts episodic language and visual memories into plans for execution. Our work focuses on memory for pretrained VLAs, studying how compact latent representations of multimodal histories should be directly supervised and jointly adapted with control.

Learning Memory Representations. Learning compact latent memory requires supervision for what information should survive compression. Reconstruction objectives preserve information from sequential experience, including visual content Wang et al. (2023), latent dynamics Hafner et al. (2019), and past actions Torne et al. (2025), while time-contrastive learning Sermanet et al. (2018) and progress prediction Zhai et al. (2025) provide signals about task progression. For latent memory, HAMLET Koo et al. (2026) uses time-contrastive initialization, while Workspace models Dashora et al. (2026) do not explicitly require the bottleneck to preserve each input stream. In contrast, we bootstrap memory to be both *complete* and *task-aware*: reconstructing visual, action, and proprioceptive histories encourages retention from every stream, while progress estimation tracks how far the task has progressed before joint VLA adaptation for control.

3 PROBLEM FORMULATION

Notation & Setup. At any real time t , the robot observes RGB camera images $I_t \in \mathcal{I}$ from cameras and proprioceptive states $q_t \in \mathcal{Q}$, which we concatenate as $o_t := [I_t, q_t]$. Let $a \in \mathcal{A}$ denote a low-level action (e.g. joint states), $\mathbf{a}_t := a_{t:t+h}$ as an h -step action chunk, and ℓ as the task description. A pre-trained VLA with parameters θ acts only on the current observation, $\mathbf{a}_t \sim \pi_\theta(\cdot | o_t, \ell)$. We denote the interaction history by $\mathcal{H}_t := (o_{\leq t}, a_{< t})$ which contains all past visual observations, proprioceptive states, and executed actions. $\mathbb{D}$ is a set of demonstrations we use to train policies.

Memory-Augmented Policies. Many manipulation tasks require information beyond current observations: two histories $\mathcal{H}_t \neq \mathcal{H}'_t$ can end at the same observation o_t yet call for different actions,

$$\pi(\mathbf{a}_t | o_t, \ell, \mathcal{H}_t) \neq \pi(\mathbf{a}_t | o_t, \ell, \mathcal{H}'_t) \quad (1)$$

For example, while a robot dusts a table, the current scene no longer shows original locations or the sweeping path, yet returning the objects and finishing the dusting depend on that. To handle such *history-dependent* steps, a memory-augmented policy π_θ^{mem} usually summarize the history into a memory representation m_t through a function f and conditions on it to generate actions:

$$m_t = f(\mathcal{H}_t, \ell), \quad \mathbf{a}_t \sim \pi_\theta^{\text{mem}}(\cdot | o_t, \ell, m_t). \quad (2)$$

The central question is therefore how to design the memory function f without restricting what it can represent, as explicit forms do. Since $\mathcal{H}_t$ grows continuously and spans heterogeneous visual, action, and proprioceptive streams, retaining it in full is neither scalable nor necessary for control. Instead, f should compress the growing history into a bounded representation that preserves the information needed for future decisions. We therefore consider the setting where f is a learned *latent memory encoder* $\mathcal{E}_\phi^{\text{mem}}$ that maps the multimodal interaction history to a fixed-size continuous representation:

$$m_t = \mathcal{E}_\phi^{\text{mem}}(\mathcal{H}_t, \ell) \in \mathbb{R}^{B \times d}, \quad (3)$$

where B does not grow with t . Our goal is a memory representation m_t that compresses the multimodal history context $\mathcal{H}_t$ without changing the action distribution:

$$\pi(\mathbf{a}_t | o_t, \ell, \mathcal{H}_t) = \pi(\mathbf{a}_t | o_t, \ell, m_t). \quad (4)$$

To approach this goal, the encoder and policy are learned from demonstrations by minimizing an objective $\mathcal{L}$ that depends on the memory:

$$\min_{\theta, \phi} \mathbb{E}_{\mathbb{D}, t} [\mathcal{L}(\mathcal{H}_t, \mathbf{a}_t; \pi_\theta, \mathcal{E}_\phi^{\text{mem}})]. \quad (5)$$

Achieving equation 4 through equation 5 raises two challenges: how to design the encoder $\mathcal{E}_\phi^{\text{mem}}$, and how to choose the objective $\mathcal{L}$.

Challenge 1: Encoding Heterogeneous Interaction Histories Online. The history combines a high-dimensional video stream with low-dimensional proprioceptive states and actions, which differ in scale and structure. The video stream is particularly hard to compress: both spatial and temporal dimensions have high redundancy, yet the small changes between them, such as object locations, matter for later decisions. Moreover, the history grows at every step while memory is queried at every control step, so re-encoding the full history adds latency that grows with the episode length.

Challenge 2: Choosing the Memory Objective. The standard choice for Eq. 5 is end-to-end optimization through action loss Shi et al. (2026), where the memory is learned only through gradients backpropagated from the policy:

$$\mathcal{L}_{\text{act}}(\theta, \phi) = \mathbb{E}_{\mathbb{D}, t} \left[-\log \pi_{\theta}^{\text{mem}}(\mathbf{a}_t \mid o_t, \ell, \mathcal{E}_{\phi}^{\text{mem}}(\mathcal{H}_t, \ell)) \right]. \quad (6)$$

This objective supervises memory only indirectly through action prediction, coupling what the bottleneck should retain with how the policy uses it for control. In particular, it provides no explicit supervision that memory preserves information from each heterogeneous history stream.

4 IMPRINT: INTERACTION MEMORY WITH PROGRESS- AND RECONSTRUCTION-INFORMED TRAINING

We address the challenges in Sec. 3 with efficient encoding and two-stage learning. We first compress visual, action, and proprioceptive histories into a fixed-size latent memory (Sec. 4.1). We then bootstrap memory with history reconstruction and progress estimation (Sec. 4.2.1), before jointly adapting the memory and VLA with action supervision for control (Sec. 4.2.2).

4.1 MULTIMODAL MEMORY ENCODING

The first challenge is to efficiently compress a growing, heterogeneous interaction history into bounded memory. We address this with two levels of compression: within stream and across streams over time. Because video dominates the history in both dimensionality and redundancy, we first compress it with a frozen causal video tokenizer Wan et al. (2025); Agarwal et al. (2026). The memory encoder then integrates the resulting visual tokens with actions and proprioceptive states over time into a fixed number of latent memory tokens.

Tokenizing the Interaction Stream. We represent interaction history through four token streams: *historical video*, *recent images*, *executed actions*, and *proprioceptive states*. Let $I_{\leq n_t}$ denote the previous observed frames available at time t . With L frames per chunk, $K_t = \lfloor n_t/L \rfloor$ chunks are complete. A frozen causal video tokenizer $\mathcal{T}_{\psi}^{\text{vid}}$ Wan et al. (2025) divides these frames $I_{\leq K_t L}$ into chunks and then tokenize them into a small set of tokens per camera. Only completed chunks are encoded, ensuring causality. The remaining frames $I_{K_t L+1:n_t}$ form the recent-image stream, tokenized by off-the-shelf DINO encoder $\mathcal{T}_{\psi}^v$ Siméoni et al. (2025) to preserve recent features z_k^v . Separate MLPs embed executed actions $a_{\leq t}$ and proprioceptive states $q_{\leq t}$ into the other two streams, with each action token encoding only actions executed before its associated observation.

Memory Bottleneck. Let e_t^{vid} , e_t^{recent} , e_t^{act} , and e_t^{state} denote the outputs of the four tokenization streams. We introduce B learned query embeddings $\mathbf{Q} \in \mathbb{R}^{B \times d}$ that read out information from the four token streams conditioned on the task description ℓ . The memory encoder computes $m_t = \mathcal{E}_{\phi}^{\text{mem}}(e_t^{\text{vid}}, e_t^{\text{recent}}, e_t^{\text{act}}, e_t^{\text{state}}; \mathbf{Q}, \ell) \in \mathbb{R}^{B \times d}$.

The learned queries are shared across episodes and conditioned on language features of ℓ , compressing the history into B memory tokens. We consider two bottlenecks with the same supervision (Sec. 4.2). Writing the four token streams collectively as e_t , they compute

$$\text{Attention: } m_t = \mathcal{E}_{\phi}^{\text{mem}}(e_t; \mathbf{Q}, \ell), \quad (7)$$

$$\text{Recurrent: } (m_t, h_t) = \mathcal{E}_{\phi}^{\text{mem}}(e_t; h_{t-1}, \mathbf{Q}, \ell). \quad (8)$$

i) *Full Attention bottleneck.* Following Perceiver Jaegle et al. (2021), the queries cross-attend to all history tokens, including the recent window. Each query revisits the full history, so storage and per-query cost grow linearly with history length. Without positional encodings, temporal order is available only through token content.

ii) *Recurrent bottleneck.* Gated delta blocks Yang et al. (2025) process each finalized chunk once as chronological action, state, and video events, tagged with position and chunk index. They update fixed-size matrix states, collectively denoted by h_t . The queries read these states and then attend to the recent window, keeping storage and per-query cost bounded independently of history length. Training uses the parallel formulation of the gated delta blocks. At inference time, The recurrent

bottleneck caches matrix states, updating them once per finalized chunk. Queries read these states and the bounded recent window, keeping per-step cost and cache size independent of history length. The attention bottleneck instead stores and revisits the growing history at each query.

4.2 DECOUPLED MEMORY SUPERVISION

Optimizing Eq. 5 jointly requires learning both what information memory should retain and how the policy should use it for control. With action supervision alone, the memory encoder is guided only indirectly through the policy, providing no explicit requirement that it preserve information from each history stream. We therefore decouple these two roles: first bootstrap memory with direct, policy-independent supervision, and then jointly adapt the memory and policy with action loss.

Memory Objective & Training Pipeline We use two complementary objectives to make memory *complete* and *task-aware* without additional annotations. *History reconstruction* requires m_t to recover past visual features, actions, and proprioceptive states, asking it to retain information from every stream. *Progress estimation* requires m_t to predict how far the task has progressed, offering global task context that reconstruction alone may not capture. Both objectives rely on a decoder $\mathcal{D}_\varphi$ with parameters φ that takes m_t as input. This naturally yields a two-stage training procedure:

$$\begin{aligned} \text{Stage 1: } \bar{\phi}, \bar{\varphi} &= \arg \min_{\phi, \varphi} \mathbb{E}_{\mathbb{D}, t} \left[\mathcal{L}_{\text{mem}} \left(\mathcal{D}_\varphi \left(\mathcal{E}_\phi^{\text{mem}}(\mathcal{H}_t, \ell), \mathbf{y}_t \right) \right), \right. \\ \text{Stage 2: } \theta^*, \phi^* &= \arg \min_{\theta, \phi} \mathbb{E}_{\mathbb{D}, t} \left[-\log \pi_\theta(\mathbf{a}_t \mid o_t, \ell, \mathcal{E}_\phi^{\text{mem}}(\mathcal{H}_t, \ell)) \right], \end{aligned} \quad (9)$$

where $\mathbf{y}_t$ represents the reconstruction targets at sampled past steps and the progress ρ_t , and $\mathcal{L}_{\text{mem}}$ is the prediction error defined in equation 10. Stage 1 trains the encoder $\mathcal{E}_\phi^{\text{mem}}$ and decoder $\mathcal{D}_\varphi$ without the policy. Stage 2 discards the decoder and initialize encoder $\mathcal{E}_\phi^{\text{mem}}$ from parameters $\bar{\phi}$ and finetunes the VLA policy π_θ jointly with the memory encoder, so that the action objective reallocates the task-agnostic memory toward decision-relevant content and teaches the policy to read it.

4.2.1 STAGE 1: MEMORY PRE-TRAINING

Following Jaegle et al. (2021), the decoder $\mathcal{D}_\varphi$ queries m_t with a learned embedding of a past step $k \leq t$ and predicts that step’s visual features $\hat{z}_k^v$, action $\hat{a}_k$, and state $\hat{q}_k$. For each step, it forms one query per visual patch and one summary query; these queries attend to each other within the step and cross-attend to m_t . The patch queries predict $\hat{z}_k^v$, and the summary query predicts $\hat{a}_k$ and $\hat{q}_k$. A separate global query predicts the progress $\hat{\rho}_t$. For K steps sampled across the history, the loss is:

$$\mathcal{L}_{\text{mem}} = \underbrace{\frac{1}{K} \sum_k \left[\lambda_v \|\hat{z}_k^v - z_k^v\|^2 + \lambda_a \|\hat{a}_k - a_k\|^2 + \lambda_q \|\hat{q}_k - q_k\|^2 \right]}_{\text{history reconstruction}} + \underbrace{\lambda_\rho (\hat{\rho}_t - \rho_t)^2}_{\text{progress estimation}}, \quad (10)$$

where $z_k^v = \mathcal{T}_\psi^v(I_k)$ are frozen, normalized patch features and $\rho_t \in [0, 1]$ is the normalized progress of the demonstration. The first three terms supervise history reconstruction at individual past steps, and the last term supervises progress estimation over the entire history.

4.2.2 STAGE 2: POLICY-AGNOSTIC MEMORY ADAPTATION

Memory Interface. Stage 1 produces a policy-independent memory encoder that can integrate with different VLAs through a lightweight action-expert adapter, following Dai et al. (2026). At each layer, action tokens x cross-attend to m_t and receive FiLM modulation Perez et al. (2018): $\tilde{x} = \text{FiLM}(x, \text{CrossAttn}(x, m_t))$. This preserves the VLA’s native inputs, context length, and conditioning, while better preserving pretrained representations than appending memory to the VLM context Dai et al. (2026). Attending to a fixed number of memory tokens adds little cost, independent of history length. We instantiate this adapter in different VLA backbones including π_0 Black et al. (2024), $\pi_{0.5}$ Intelligence et al. (2025), and MolmoAct2 Fang et al. (2026) (App. ??).

Joint Adaptation. Following Stage 2 of equation 9, we discard the decoder and initialize the policy from θ_0 and the encoder from $\bar{\phi}$. The adapter is initialized near zero, so training starts close to the original VLA. We then optimize the action objective equation 6 with the VLA’s native loss (e.g.,

flow matching), updating the encoder, adapter, and VLA. Because the bootstrapped memory already carries critical information from the history, the policy can easily rely on that to generate correct actions, and the action gradient in turn further refines memory for control.

5 EXPERIMENTS

We evaluate whether **IMPRINT** improves manipulation across tasks with different memory requirements while keeping memory compact and efficient. We first compare **IMPRINT** against existing memory approaches in simulation and real world, on task success and inference latency in Sec. 5.2. We then analyze *memory encoding*, how memory capacity and recurrent compression affect memory representation and task performance in Sec. 5.3; and *memory learning*, how memory bootstrapping and adaptation affect control, as well as its flexibility to different VLA backbones in Sec. 5.4.

5.1 EXPERIMENTAL SETUP

Benchmarks and Tasks. We evaluate **IMPRINT** on two simulation benchmarks and three real-world manipulation tasks. In simulation, we use all 9 memory-dependent tasks from RMBench Chen et al. (2026b) and a customized 12-task benchmark built on RoboCasa Nasiriany et al. (2024) and RememBench Shah et al. (2026a), containing 6 memory-dependent and 6 memory-free tasks with diverse objects and scenes. This setup evaluates both the benefits of memory and its impact on tasks without memory necessity. In the real world, we evaluate three memory-dependent tasks on a bimanual YAM platform: *Dust Table*, *Scan Bottle*, and *Fold Cloth*, covering spatial memory of object locations, task-progress tracking of scanning, and learning from in-context examples of varied folding styles. Task details are provided in Appendix ??.

Baselines. We compare against three VLAs, π_0 , $\pi_{0.5}$, MolmoAct2 without memory and three categories of memory-augmented policies from prior work: sampling visual frames with heuristics **Uniform Sampling** and **EventVLA** Dai et al. (2026); Yang et al. (2026a), hierarchical memory methods with textual summary and keyframes **Mem-0** and **MemER** Sridhar et al. (2026); Chen et al. (2026b), and learned latent-memory methods **HAMLET** and **MemoryVLA** Shi et al. (2026); Koo et al. (2026). We additionally conduct controlled ablations of encoder, memory capacity, bootstrapping objectives, and adaptation with VLAs. Details of all baselines and ablations are in Appendix. ??

Training and Evaluation. Unless otherwise specified, we use $\pi_{0.5}$ as base policy and $B = 256$ memory tokens. We report results over 100 trials per task in Rmbench and 50 trials per task in Robocasa. Our primary metric is overall task success. Following Team et al. (2025), we plot the smoothed distribution of task success symmetrically, with width indicating density and a horizontal line marking the mean. We separately report memory and manipulation success to distinguish different failure modes in Appendix. ?? with other evaluation details.

5.2 **IMPRINT** OUTPERFORMS EXISTING MEMORY-AUGMENTED VLAS WITH BETTER INFERENCE EFFICIENCY

We first evaluate the overall effectiveness of **IMPRINT** across simulation and real-world manipulation. We compare against the base VLA and existing heuristic, hierarchical and latent memory approaches, measuring both task success and inference latency.

Simulation Results. Figure 1 compares **IMPRINT** with heuristic, hierarchical, and latent memory approaches on the 9 memory-dependent RMBench tasks. Heuristic selection or sampling of sparse visual frames helps over the no-memory $\pi_{0.5}$ baseline (25.4%) to 67.8% but fails on tasks that needs denser information (e.g.,

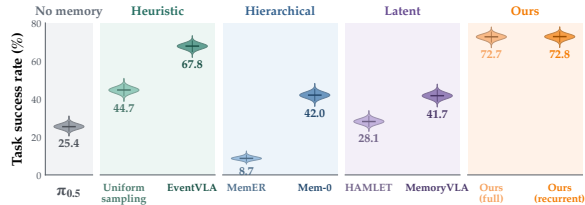


Figure 1: **Average Task Performance on Rmbench** (press button). Hierarchical methods achieve only maximum of 42.0% success, caused by the loss of information in the reduced subset of memories and imprecision near the subtask boundaries. Prior latent memories achieves at most 41.7%, but either operate primarily on visual history or lack

direct supervision to preserve information from each interaction stream. In contrast, **IMPRINT** bootstraps a multi stream latent memory to retain visual, action, and proprioceptive history while tracking task progress, reaching 72.8% success.

Figure 2 examines whether **IMPRINT** helps with tasks that do not require history. Heuristic memory can improve memory-dependent performance about 16% but substantially degrades memory-free tasks by 22%, indicating that simply exposing the VLA to additional history can interfere with behaviors already solvable from the current observation. In contrast, recurrent **IMPRINT** raises memory-dependent success to 65.3% while slightly improving memory-free performance to 68.0%, achieving 66.7% overall. This suggests that compressing history into a learned latent representation allows the policy to exploit memory when needed without disrupting memory-free behaviors.

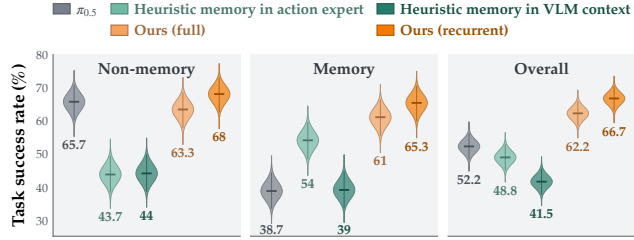


Figure 2: Average Task Performance For Memory-dependent and Memory-free Tasks in Robocasa

Real-World Results. Figure 3 shows that the gains of **IMPRINT** transfer to real-world manipulation. **IMPRINT** achieves 80%, 75%, and 72% success across the three tasks, consistently outperforming all baselines. Compared with the strongest baseline, uniform sampling (47–50%), **IMPRINT** improves average success by 26.7%. The consistent gains over both explicit and latent memory baselines suggest that directly learning a compact multimodal memory is also effective under real-world memory tasks that requires spatial memory of objects, task progress tracking of scanning and in-context learning of folding styles.

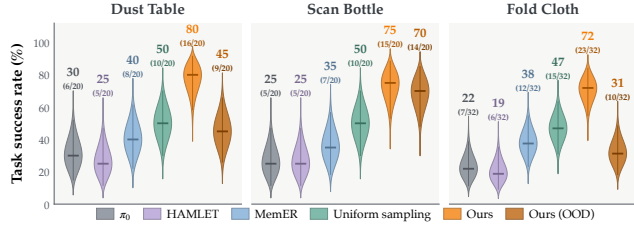


Figure 3: Real World Task Performance Across 3 Tasks

Real-World OOD Evaluation. We additionally test **IMPRINT** on a small set of OOD variations of unseen objects, where it achieves 45%, 70%, and 31% success across these tasks. While performance decreases under these shifts, particularly on *Dust Table* and *Fold Cloth* due to longer horizon and higher precision requirement, this small robustness experiment provides initial evidence that the learned memory can transfer beyond the training configurations.

Success and Inference Efficiency. Figure 4 compares task success and average inference latency per 50-step action chunk under L40S GPU. Existing memory methods incur either limited performance gains or substantial latency: HAMLET and MemoryVLA reach 28.1% and 41.7% success at 151 and 194 ms, while EventVLA reaches 67.8% but requires 960 ms due to long context to Qwen backbone. Hierarchical methods introduce even more latency with several seconds from high-level VLMs. In contrast, recurrent **IMPRINT** achieves the highest success (72.8%) at 143.2 ms, adding only 19.6 ms over the no-memory $\pi_{0.5}$ and dominating all evaluated memory baselines in both success and latency.

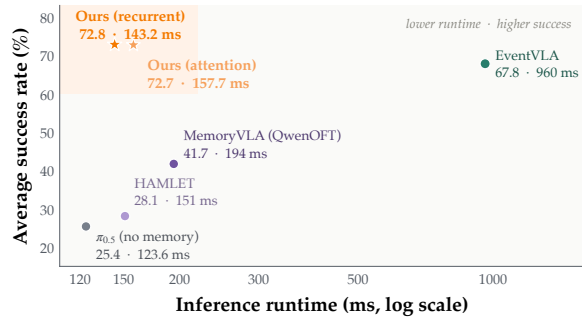


Figure 4: Inference Time vs Success on RMBench .

5.3 MEMORY CAPACITY AND RECURRENT COMPRESSION IMPROVE MEMORY ENCODING

We next study the encoding challenge from Sec. 3: compressing a growing interaction history into a continuous latent representation. We examine two properties of the memory bottleneck: its *capacity*,

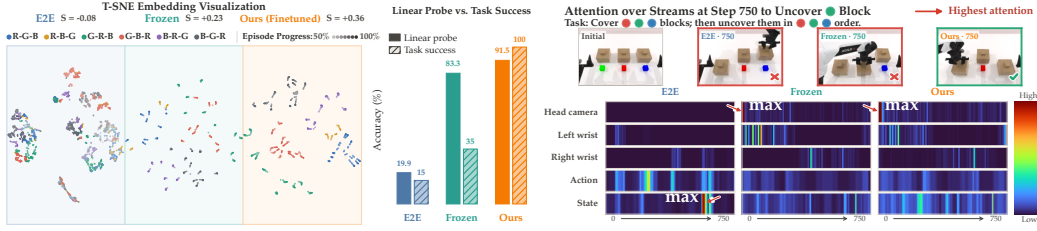


Figure 7: **Analysis of Learned Memory Bottleneck.** We compare E2E approach, frozen bootstrapped memory, and our jointly adapted memory using t-SNE embeddings, linear probes, and attention visualization on *Cover Block* task in RMBench.

which controls how much historical information can be retained, and its *update mechanism*, which determines how efficiently history is accumulated over time.

More Capacity Preserves More Useful Context.

We vary the number of latent memory tokens from 4 to 256 while keeping the encoder and training procedure fixed. Performance improves with capacity across all three evaluations. RMBench increases from 67.0% to 72.7%, while the memory subset of RoboCasa improves from 54.0% to 61.0%. RoboCasa non-memory performance remains around 57% up to 64 tokens, but rises to 63.3% at 256 tokens. These results reveal a natural scaling trend with memory capacity, with larger bottlenecks consistently yielding stronger downstream performance

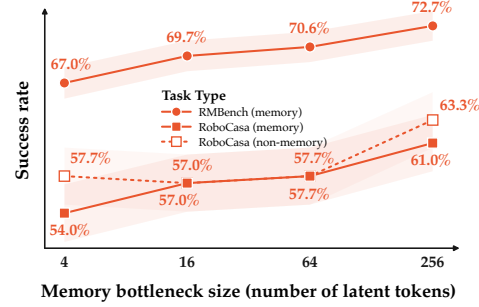


Figure 5: **Memory Capacity versus Performance on RMBench and RoboCasa.**

Recurrent Compression Improves Encoding Efficiency. Beyond capacity, the memory encoder must also accumulate a growing history efficiently. We therefore compare our recurrent encoder with the standard-attention variant under the same 256-token memory capacity. On RMBench, both achieve nearly identical success (72.8% vs. 72.7%), while on RoboCasa recurrent encoding improves overall success from 62.2% to 66.7%. It also reduces total inference latency from 157.7 ms to 143.2 ms per action chunk. Thus, recurrent compression lowers the inference latency while matching or improving downstream performance.

5.4 MEMORY BOOTSTRAPPING AND JOINT ADAPTATION IMPROVE PERFORMANCE ACROSS VLA BACKBONES

Bootstrapping and Joint Adaptation Are Both Important.

We compare end-to-end action supervision (E2E) in Eq. 6, frozen bootstrapped memory without adaptation, and bootstrapping followed by joint memory-VLA adaptation. As shown in Fig. 6 left panel, success improves from 50.8% (E2E) to 58.8% with bootstrapping alone, and further to 72.7% with joint adaptation. This shows that both memory bootstrapping and subsequent adaptation for action generation are important.

Bootstrapping Produces More Informative Memory.

Figure 7 shows that bootstrapping improves linear-probe accuracy from 19.9% to 83.3%, reaching 91.5% after joint adaptation, while t-SNE structure improves from $S = -0.08$ to 0.36. Attention visualizations show that bootstrapped memory attends to the color order in the initial frames of head camera, whereas E2E pays wrong attention to state history. However, only joint adaptation enables **IMPRINT** to use this information to uncover the blocks in the correct red-to-green order,

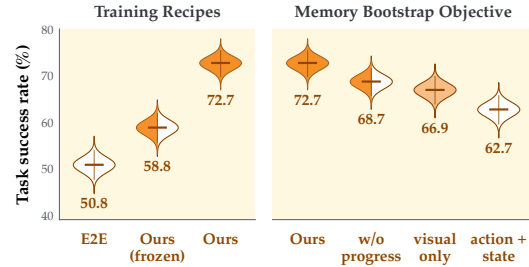


Figure 6: **Ablations of Memory Learning.**

showing that bootstrapping improves what memory retains while adaptation improves how it is used for action generation.

Reconstruction and Progress Estimation Are Complementary Objectives. We next ablate different bootstrapping objectives while keeping downstream adaptation fixed in Figure. 6 right panel. The full combination of visual, action, and state reconstruction with progress estimation achieves 72.7% success. Removing progress estimation reduces performance to 68.7%, while using only visual reconstruction or only action and state reconstruction further decreases success to 66.9% and 62.7%, respectively. These results show that effective memory formation benefits from both history retention and task-progress estimation.

The Same Learning Pipeline Transfers across VLA Backbones. Because Stage 1 bootstrapping is independent of the downstream policy, we apply the same memory pipeline and adaptation procedure to three different VLA backbones. As shown in Fig. 8, memory augmentation improves success from 25.4% to 72.7% for $\pi_{0.5}$, from 17.4% to 63.0% for π_0 , and from 18.8% to 60.0% for MolmoAct2. The consistent gains across different base policies show that the proposed memory bootstrapping and joint-adaptation recipe is not tied to a particular VLA architecture.

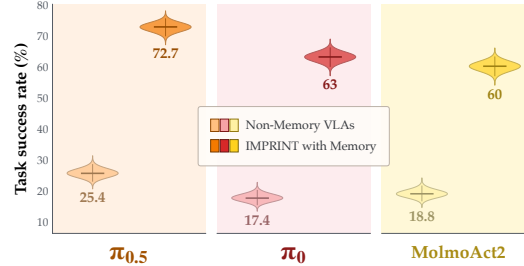


Figure 8: IMPRINT Consistently Improves Different VLA Backbones.

6 LIMITATIONS AND CONCLUSION

Limitations. A bounded latent memory is less interpretable than explicit keyframes or language summaries, making it harder to diagnose failures. Our memory is also bootstrapped from the same limited demonstration data used for policy learning, which may restrict the diversity of historical patterns it can capture. Scaling Stage 1 to larger and more diverse interaction datasets, including both successful and failed trajectories, is therefore an important direction. Finally, we evaluate multiple VLA backbones and limited real-world distribution shifts, broader transfer across embodiments and open-world settings remains to be studied.

Conclusion. We presented IMPRINT, a policy-agnostic latent memory framework for VLAs. IMPRINT compresses visual, action, and proprioceptive histories into bounded latent memory, bootstraps the representation to preserve interaction history and track task progress, and then jointly adapts memory with the VLA for action generation. Across simulation and real-world manipulation, IMPRINT outperforms base VLAs and existing memory approaches while maintaining low inference overhead, supporting latent memory as a reusable component for generalist robot policies.

AI USE STATEMENT

In this work, we used generative AI tools to help implement parts of the training and evaluation code. We have not used generative AI tools to design experiments, generate data, formulate mathematical claims, or interpret results. Additionally, we used generative AI tools to improve the writing and presentation of the paper, including editing for clarity, suggesting its structure and title. All AI-assisted code was reviewed and tested by the authors, and all AI-assisted text was revised by the authors. We take responsibility for the final content of this work.

REPRODUCIBILITY STATEMENT

We will publicly release our code, model checkpoints, and data, including the customized RoboCasa benchmark and the preprocessed training data, upon publication. The problem formulation and method are described in Sec. 3 and Sec. 4. Architecture details, Stage-1 and Stage-2 training hyperparameters, and input processing for all streams are provided in App. ??, and policy-specific integration details for π_0 , $\pi_{0.5}$, and MolmoAct2 are given in App. ??. The benchmark tasks, data collection, and evaluation protocols are described in [Sec. X / App. Y].

REFERENCES

- Niket Agarwal, Arslan Ali, Jon Allen, Martin Antolini, Adeline Aubame, Alisson Azzolini, Junjie Bai, Maciej Bala, Yogesh Balaji, Josh Bapst, et al. Cosmos 3: Omnimodal world models for physical ai. *arXiv preprint arXiv:2606.02800*, 2026.
- Anuva Banwasi, William Muckelroy III, Priya Sundareshan, Linfeng Zhao, Jeannette Bohg, and Cherie Ho. Messymem: Learning-from-doing memory for mobile manipulation. *arXiv preprint arXiv:2609.15976*, 2026.
- Johan Bjorck, Fernando Castañeda, Nikita Cherniadev, Xingye Da, Runyu Ding, Linxi Fan, Yu Fang, Dieter Fox, Fengyuan Hu, Spencer Huang, et al. Gr00t n1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025.
- Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al. pi: A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.
- Tianxing Chen, Yue Chen, Zixuan Li, Junyuan Tang, Kailun Su, Weijie Wan, Baijun Chen, Haoran Lu, Haowen Yan, Honghao Su, et al. RoboDojo: A unified sim-and-real benchmark for comprehensive evaluation of generalist robot manipulation policies. *arXiv preprint arXiv:2607.04434*, 2026a.
- Tianxing Chen, Yuran Wang, Mingleyang Li, Yan Qin, Hao Shi, Zixuan Li, Yifan Hu, Yingsheng Zhang, Kaixuan Wang, Yue Chen, et al. Rmbench: Memory-dependent robotic manipulation benchmark with insights into policy design. *arXiv preprint arXiv:2603.01229*, 2026b.
- Yinpei Dai, Hongze Fu, Jayjun Lee, Yuejiang Liu, Haoran Zhang, Jianing Yang, Chelsea Finn, Nima Fazeli, and Joyce Chai. Robomme: Benchmarking and understanding memory for robotic generalist policies. *Proceedings of the 43 rd International Conference on Machine Learning (ICML)*, 2026.
- Nitish Dashora, Douglas Chen, Idan Shenfeld, John Marangola, Pulkit Agrawal, and Max Simchowitz. Workspace models: Lightweight robotic memory via saliency-driven supervision, 2026. URL <https://arxiv.org/abs/2609.20820>.
- Haoquan Fang, Jiafei Duan, Donovan Clay, Sam Wang, Shuo Liu, Weikai Huang, Xiang Fan, Wei-Chuan Tsai, Shirui Chen, Yi Ru Wang, et al. Molmoact2: Action reasoning models for real-world deployment. *arXiv preprint arXiv:2605.02881*, 2026.
- Yihuai Gao, Jinyun Liu, Shuang Li, and Shuran Song. Gated memory policy. *arXiv preprint arXiv:2604.18933*, 2026.
- Danijar Hafner, Timothy Lillicrap, Ian Fischer, Ruben Villegas, David Ha, Honglak Lee, and James Davidson. Learning latent dynamics for planning from pixels. In *International conference on machine learning*, pp. 2555–2565. PMLR, 2019.
- Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, et al. pi 0.5: a vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.
- Andrew Jaegle, Felix Gimeno, Andy Brock, Oriol Vinyals, Andrew Zisserman, and Joao Carreira. Perceiver: General perception with iterative attention. In *International conference on machine learning*, pp. 4651–4664. PMLR, 2021.
- Hanxiao Jiang, Binghao Huang, Ruihai Wu, Zhuoran Li, Shubham Garg, Hooshang Nayyeri, Shenlong Wang, and Yunzhu Li. Roboexp: Action-conditioned scene graph via interactive exploration for robotic manipulation. In *Conference on Robot Learning (CoRL)*, 2024.
- Yunfan Jiang, Yevgen Chebotar, Ruijie Zheng, Fengyuan Hu, Yunhao Ge, Jimmy Wu, Tianyuan Dai, Scott Reed, Li Fei-Fei, Yuke Zhu, and Linxi ”Jim” Fan. Robottt: Context scaling for robot policies. *arXiv preprint arXiv: 2607.15275*, 2026.

- Myungkyu Koo, Daewon Choi, Taeyoung Kim, Kyungmin Lee, Changyeon Kim, Younggyo Seo, and Jinwoo Shin. Hamlet: Switch your vision-language-action model into a history-aware policy. In *International Conference on Learning Representations*, volume 2026, pp. 101537–101558, 2026.
- Peiqi Liu, Zhanqiu Guo, Mohit Warke, Soumith Chintala, Chris Paxton, Nur Muhammad Mahi Shafiullah, and Lerrel Pinto. Dynamem: Online dynamic spatio-semantic memory for open world mobile manipulation. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 13346–13355. IEEE, 2025.
- Max Sobol Mark, Jacky Liang, Maria Attarian, Chuyuan Fu, Debidatta Dwibedi, Dhruv Shah, and Aviral Kumar. Bpp: Long-context robot imitation learning by focusing on key history frames. *arXiv preprint arXiv:2602.15010*, 2026.
- Soroush Nasiriany, Abhiram Maddukuri, Lance Zhang, Adeet Parikh, Aaron Lo, Abhishek Joshi, Ajay Mandlekar, and Yuke Zhu. Robocasa: Large-scale simulation of everyday tasks for generalist robots. *arXiv preprint arXiv:2406.02523*, 2024.
- Ethan Perez, Florian Strub, Harm De Vries, Vincent Dumoulin, and Aaron Courville. Film: Visual reasoning with a general conditioning layer. In *Proceedings of the AAAI conference on artificial intelligence*, volume 32, 2018.
- Jianing Qian, Qinhe Peng, Emmanuel Panov, Leonor Fermoselle, Dinesh Jayaraman, Bernadette Bucher, and Tarik Kelestemur. Expanding spatial and temporal context for robotic imitation learning with scene graphs. 2026.
- Pierre Sermanet, Corey Lynch, Yevgen Chebotar, Jasmine Hsu, Eric Jang, Stefan Schaal, Sergey Levine, and Google Brain. Time-contrastive networks: Self-supervised learning from video. In *2018 IEEE international conference on robotics and automation (ICRA)*, pp. 1134–1141. IEEE, 2018.
- Rutav Shah, Rajat Kumar Jenamani, Xiaohan Zhang, Lingfeng Sun, Roberto Martín-Martín, Yuke Zhu, Deva Ramanan, and Karl Schmeckpeper. Scaling short-term memory of visuomotor policies for long-horizon tasks. *arXiv preprint arXiv:2606.16178*, 2026a.
- Rutav Shah, Yisu Li, Femi Bello, Yuke Zhu, and Roberto Martín-Martín. Memory retrieval in visuomotor policies for long-horizon robot control. In *Proceedings of Robotics: Science and Systems*, 2026b.
- Hao Shi, Bin Xie, Yingfei Liu, Lin Sun, Fengrong Liu, Tiancai Wang, Erjin Zhou, Haoqiang Fan, Xiangyu Zhang, and Gao Huang. Memoryvla: Perceptual-cognitive memory in vision-language-action models for robotic manipulation. In *International Conference on Learning Representations*, volume 2026, pp. 18567–18602, 2026.
- Oriane Siméoni, Huy V Vo, Maximilian Seitzer, Federico Baldassarre, Maxime Oquab, Cijo Jose, Vasil Khalidov, Marc Szafraniec, Seungeun Yi, Michaël Ramamonjisoa, et al. Dinov3. *arXiv preprint arXiv:2508.10104*, 2025.
- Ajay Sridhar, Jennifer Pan, Satvik Sharma, and Chelsea Finn. Scaling up memory for robotic control via experience retrieval. In *International Conference on Learning Representations*, volume 2026, pp. 97142–97166, 2026.
- TRI LBM Team, Jose Barreiros, Andrew Beaulieu, Aditya Bhat, Rick Cory, Eric Cousineau, Hongkai Dai, Ching-Hsin Fang, Kunitatsu Hashimoto, Muhammad Zubair Irshad, Masha Itkina, Naveen Kuppaswamy, Kuan-Hui Lee, Katherine Liu, Dale McConachie, Ian McMahon, Haruki Nishimura, Calder Phillips-Grafflin, Charles Richter, Paarth Shah, Krishnan Srinivasan, Blake Wulfe, Chen Xu, Mengchao Zhang, Alex Alspach, Maya Angeles, Kushal Arora, Victor Campagnolo Guizilini, Alejandro Castro, Dian Chen, Ting-Sheng Chu, Sam Creasey, Sean Curtis, Richard Denitto, Emma Dixon, Eric Dusel, Matthew Ferreira, Aimee Goncalves, Grant Gould, Damrong Guoy, Swati Gupta, Xuchen Han, Kyle Hatch, Brendan Hathaway, Allison Henry, Hillel Hochshtein, Phoebe Horgan, Shun Iwase, Donovan Jackson, Siddharth Karamcheti, Sedrick Keh, Joseph Masterjohn, Jean Mercat, Patrick Miller, Paul Mitiguy, Tony Nguyen, Jeremy

- Nimmer, Yuki Noguchi, Reko Ong, Aykut Onol, Owen Pfannenstiehl, Richard Poyner, Leticia Priebe Mendes Rocha, Gordon Richardson, Christopher Rodriguez, Derick Seale, Michael Sherman, Mariah Smith-Jones, David Tago, Pavel Tokmakov, Matthew Tran, Basile Van Hoorick, Igor Vasiljevic, Sergey Zakharov, Mark Zolotas, Rares Ambrus, Kerri Fetzer-Borelli, Benjamin Burchfiel, Hadas Kress-Gazit, Siyuan Feng, Stacie Ford, and Russ Tedrake. A careful examination of large behavior models for multitask dexterous manipulation. 2025. URL <https://arxiv.org/abs/2507.05331>.
- Marcel Torne, Andy Tang, Yuejiang Liu, and Chelsea Finn. Learning long-context diffusion policies via past-token prediction. *arXiv preprint arXiv:2505.09561*, 2025.
- Marcel Torne, Karl Pertsch, Homer Walke, Kyle Vedder, Suraj Nair, Brian Ichter, Allen Z Ren, Haohuan Wang, Jiaming Tang, Kyle Stachowicz, et al. Mem: Multi-scale embodied memory for vision language action models. *arXiv preprint arXiv:2603.03596*, 2026.
- Team Wan, Ang Wang, Baole Ai, Bin Wen, Chaojie Mao, Chen-Wei Xie, Di Chen, Feiwu Yu, Haiming Zhao, Jianxiao Yang, et al. Wan: Open and advanced large-scale video generative models. *arXiv preprint arXiv:2503.20314*, 2025.
- Limin Wang, Bingkun Huang, Zhiyu Zhao, Zhan Tong, Yinan He, Yi Wang, Yali Wang, and Yu Qiao. Videomae v2: Scaling video masked autoencoders with dual masking. In *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 14549–14560. IEEE, 2023.
- Ganlin Yang, Zhangzheng Tu, Yuqiang Yang, Sitong Mao, Junyi Dong, Tianxing Chen, Jiaqi Peng, Jing Xiong, Jiafei Cao, Jifeng Dai, et al. Eventvla: Event-driven visual evidence memory for long-horizon vision-language-action policies. *arXiv preprint arXiv:2606.20092*, 2026a.
- Sizhe Yang, Juncheng Mu, Tianming Wei, Chenhao Lu, Xiaofan Li, Linning Xu, Zhengrong Xue, Zhecheng Yuan, Dahua Lin, Jiangmiao Pang, et al. Memorywam: Efficient world action modeling with persistent memory. *arXiv preprint arXiv:2606.20562*, 2026b.
- Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving mamba2 with delta rule. In *International Conference on Learning Representations*, volume 2025, pp. 29687–29707, 2025.
- Yihan Zeng, Minghao Ye, Yiyuan Chen, Yide Shentu, Philipp Wu, Zike Yan, and Zhongyu Li. Kemo: Event-driven keyframe memory for long-horizon robot manipulation with vla policies. *arXiv preprint arXiv:2606.23589*, 2026.
- Shaopeng Zhai, Qi Zhang, Tianyi Zhang, Fuxian Huang, Haoran Zhang, Ming Zhou, Shengzhe Zhang, Litao Liu, Sixu Lin, and Jiangmiao Pang. A vision-language-action-critic model for robotic real-world reinforcement learning. *arXiv preprint arXiv:2509.15937*, 2025.
- Junbo Zhang and Kaisheng Ma. Benchmarking long-horizon mobile manipulation in multi-room dynamic environments. In *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 83–89. IEEE, 2025.
- Sizhe Zhao, Haozhe Xie, Weiye Zhao, Chenchu Zhang, Huan Wang, Chenyang Wang, Qinglin Liu, and Shengping Zhang. Memory as Plans: world-action modeling with memory-grounded planning. *arXiv 2609.11561*, 2026.

A APPENDIX

You may include other additional sections here.